

COMPLETE GUIDE

Εργαστηριακό Θέμα - Ενότητα Α

Turtlebot3 Motion Controller

Unicycle Robot - Closed Loop Position Control

Πανεπιστήμιο Θεσσαλίας

Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Προγραμματισμός Ρομποτικών Συστημάτων

Περιβάλλον:	Ubuntu 20.04 LTS + ROS Noetic + Gazebo
Ρομπότ:	Turtlebot3 Waffle (Unicycle Model)
Γλώσσα:	Python 3
Παραδοτέο:	turtlebot3_motion_controller.py
Waypoints:	[2,2,0] - [4,4,0] - [6,6,0] - [8,8,0]

ΠΕΡΙΕΧΟΜΕΝΑ

PHASE 1 - Εγκατάσταση Ubuntu 20.04 στο VirtualBox

- Download Ubuntu ISO
- Δημιουργία Virtual Machine
- Εγκατάσταση Ubuntu
- Ρυθμίσεις μετά την εγκατάσταση

PHASE 2 - Εγκατάσταση ROS Noetic

- Προσθήκη ROS repository
- Εγκατάσταση ROS πακέτων
- Ρύθμιση environment
- Δημιουργία catkin workspace

PHASE 3 - Εγκατάσταση Turtlebot3 Simulator

- Clone turtlebot3 packages
- Build workspace
- Εκκίνηση simulation
- Εξερεύνηση ROS topics

PHASE 4 - Ανάπτυξη Motion Controller

- Κατανόηση ελεγκτή unicycle
- Δομή κώδικα Python/ROS
- Υλοποίηση controller node
- Waypoint navigation

PHASE 5 - Δοκιμή και Αποστολή

- Εκτέλεση και debugging
- Έλεγχος waypoint navigation
- Προετοιμασία παράδοσης
- Αποστολή email

ΕΓΚΑΤΑΣΤΑΣΗ UBUNTU 20.04 ΣΤΟ VIRTUALBOX

Ρύθμιση virtual machine και εγκατάσταση λειτουργικού συστήματος

Βήμα 1.1: Download Ubuntu 20.04.6 LTS ISO

Κατέβασε το Ubuntu 20.04.6 LTS (Focal Fossa) Desktop ISO image από:

<https://releases.ubuntu.com/20.04/>

Επέλεξε το αρχείο **ubuntu-20.04.6-desktop-amd64.iso** (περίπου 4.1 GB).

TIP: Βεβαιώσου ότι κατεβάζεις τη 20.04 (Focal Fossa) και ΟΧΙ νεότερη έκδοση. Το ROS Noetic απαιτεί αυτή ακριβώς την έκδοση.

Βήμα 1.2: Εγκατάσταση VirtualBox (αν δεν το έχεις)

Κατέβασε το VirtualBox από: <https://www.virtualbox.org/wiki/Downloads>

Εγκατέστησέ το στο host σου (Windows/Mac/Linux).

Κατέβασε επίσης το **VirtualBox Extension Pack** για υποστήριξη USB κλπ.

Βήμα 1.3: Δημιουργία Virtual Machine

Στο VirtualBox πάτα **New** και ρύθμισε τα εξής:

Ρύθμιση	Τιμή	Σημείωση
Name	Ubuntu-ROS-Noetic	Ό,τι θέλεις
Type / Version	Linux / Ubuntu (64-bit)	
RAM	4096 MB (4 GB) minimum	8 GB προτείνεται
CPU Cores	2 cores minimum	4 cores προτείνεται
Hard Disk	50 GB minimum (VDI, Dynamic)	ROS+Gazebo θέλει χώρο!
Video Memory	128 MB	Για Gazebo 3D
3D Acceleration	Enable	Display > Screen
Network	NAT ή Bridged	Για internet πρόσβαση

Βήμα 1.4: Εγκατάσταση Ubuntu στο VM

Ξεκίνα το VM, επέλεξε το ISO αρχείο ως boot disk.

Ακολούθα τον installer: Γλώσσα > Install Ubuntu > Normal Installation.

Επέλεξε **"Erase disk and install Ubuntu"** (είναι virtual disk, δεν πειράζει).

Βάλε username/password και περίμενε να ολοκληρωθεί (~15-20 λεπτά).

Κάνε **Restart** μόλις ζητηθεί.

ΠΡΟΣΟΧΗ: Μετά την εγκατάσταση, αφαιρέσε το ISO: **Devices > Optical Drives > Remove disk.**

Βήμα 1.5: Μετά την εγκατάσταση - Βασικές Ρυθμίσεις

Εκτέλεσε τις παρακάτω εντολές στο terminal:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential dkms linux-headers-$(uname -r)

# VirtualBox Guest Additions:
# Menu: Devices > Insert Guest Additions CD Image
cd /media/$USER/VBox*/
sudo ./VBoxLinuxAdditions.run
sudo reboot
```

TIP: Τα Guest Additions δίνουν shared clipboard, drag-and-drop, και auto-resize.

Βήμα 2.1: Ρύθμιση sources.list και keys

Πρόσθεσε το ROS repository στο σύστημα:

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc) main" >
/etc/apt/sources.list.d/ros-latest.list'

sudo apt install -y curl
curl -s https://raw.githubusercontent.com/ros/rosdistro/master/ros.asc | sudo apt-key
add -

sudo apt update
```

Βήμα 2.2: Εγκατάσταση ROS Noetic Full Desktop

Εγκατέστησε την πλήρη έκδοση με Gazebo:

```
sudo apt install -y ros-noetic-desktop-full
```

TIP: Η εγκατάσταση μπορεί να πάρει 15-30 λεπτά ανάλογα με τη σύνδεση.

Βήμα 2.3: Ρύθμιση Environment

Πρόσθεσε το ROS setup στο .bashrc ώστε να φορτώνεται αυτόματα:

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Βήμα 2.4: Εγκατάσταση εξαρτήσεων και εργαλείων

Εγκατέστησε τα απαραίτητα build tools:

```
sudo apt install -y python3-rosdep python3-rosinstall python3-rosinstall-generator
python3-wstool
sudo apt install -y python3-catkin-tools
sudo rosdep init
rosdep update
```

Βήμα 2.5: Δημιουργία Catkin Workspace

Δημιούργησε τον χώρο εργασίας:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/
catkin_make
echo "source ~/catkin_ws/devel/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

TIP: Κάθε φορά που ανοίγεις νέο terminal, το workspace θα είναι ήδη sourced.

Βήμα 2.6: Επιβεβαίωση εγκατάστασης ROS

Τρέξε τις εξής εντολές για να βεβαιωθείς ότι όλα δουλεύουν:

```
# Σε ένα terminal:  
roscore  
  
# Σε δεύτερο terminal:  
rostopic list  
# Πρέπει να δεις /rosout και /rosout_agg  
  
# Κλείσε το roscore με Ctrl+C
```

ΕΓΚΑΤΑΣΤΑΣΗ TURTLEBOT3 SIMULATOR

Clone, build και εκκίνηση του Turtlebot3 στο Gazebo

Βήμα 3.1: Εγκατάσταση Turtlebot3 ROS Packages

Εγκατέστησε τα βασικά πακέτα του turtlebot3:

```
sudo apt install -y ros-noetic-turtlebot3-msgs
sudo apt install -y ros-noetic-turtlebot3
```

Βήμα 3.2: Clone Turtlebot3 Simulations (noetic branch)

Κατέβασε τα simulation πακέτα στο workspace:

```
cd ~/catkin_ws/src
git clone -b noetic https://github.com/ROBOTIS-GIT/turtlebot3_simulations.git
cd ~/catkin_ws
catkin_make
```

ΠΡΟΣΟΧΗ: Βεβαιώσου ότι χρησιμοποιείς το branch noetic (-b noetic)!

Βήμα 3.3: Ρύθμιση μοντέλου Turtlebot3

Όρισε το μοντέλο ως waffle (απαίτηση εργασίας):

```
echo "export TURTLEBOT3_MODEL=waffle" >> ~/.bashrc
source ~/.bashrc

# Επιβεβαίωση:
echo $TURTLEBOT3_MODEL
# Πρέπει να εμφανίσει: waffle
```

Βήμα 3.4: Εκκίνηση Simulation (Empty World)

Ξεκίνα το Gazebo simulation:

```
roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```

TIP: Η πρώτη φορά μπορεί να πάρει λίγο χρόνο να φορτώσει τα μοντέλα. Μην πανικοβάλλεσαι!

Βήμα 3.5: Εξερεύνηση ROS Topics (ΣΗΜΑΝΤΙΚΟ!)

Σε **νέο terminal**, τρέξε τις εξής εντολές ενώ τρέχει το Gazebo:

```
# Δες όλα τα topics:
rostopic list

# ΣΗΜΑΝΤΙΚΑ TOPICS:
# /odom      --> θέση και προσανατολισμός ρομπότ (nav_msgs/Odometry)
# /cmd_vel   --> Εντολές ταχύτητας (geometry_msgs/Twist)

# Δες πληροφορίες ενός topic:
rostopic info /odom
rostopic info /cmd_vel
```

```
# Δες live δεδομένα:  
rostopic echo /odom
```

Topic	Type	Χρήση στον Controller
/odom	nav_msgs/Odometry	SUBSCRIBE - Διαβάζουμε θέση (x, y) και γωνία (yaw) του ρομπότ
/cmd_vel	geometry_msgs/Twist	PUBLISH - Στέλνουμε linear.x (ταχύτητα) και angular.z (στροφή)

Βήμα 4.1: Κατανόηση Ελεγκτή Unicycle Robot

Το turtlebot3 waffle μοντελοποιείται ως unicycle robot. Ο ελεγκτής χρησιμοποιεί:

Σφάλμα θέσης: Ευκλείδεια απόσταση μεταξύ τρέχουσας και επιθυμητής θέσης.

Σφάλμα γωνίας: Διαφορά μεταξύ τρέχοντος heading και γωνίας προς τον στόχο.

Μαθηματικό Μοντέλο Ελεγκτή:

$$e_x = x_{goal} - x_{current}$$

$$e_y = y_{goal} - y_{current}$$

$$\rho = \sqrt{e_x^2 + e_y^2} \text{ (απόσταση από στόχο)}$$

$$\alpha = \text{atan2}(e_y, e_x) - \theta \text{ (γωνία προς στόχο - τρέχον heading)}$$

Εντολές ελέγχου:

$$v = K_{\rho} * \rho \text{ (γραμμική ταχύτητα)}$$

$$\omega = K_{\alpha} * \alpha \text{ (γωνιακή ταχύτητα)}$$

Κέρδη (gains): $K_{\rho} = 0.3$, $K_{\alpha} = 0.8$ (ρυθμίστε αν χρειαστεί)

Κριτήριο τερματισμού: $\rho < 0.1 \text{ m}$ (φτάσαμε στο waypoint)

Γιατί ο επιθυμητός προσανατολισμός είναι πάντα μηδέν;

Η διαδρομή $[2,2] > [4,4] > [6,6] > [8,8]$ είναι ευθεία γραμμή στη διαγώνιο ($y=x$), οπότε κατά τη διαδοχική κίνηση ο τελικός προσανατολισμός δεν χρειάζεται συγκεκριμένη τιμή - μας αρκεί να φτάσουμε στο σημείο. Ωστόσο, ο ελεγκτής μπορεί να επεκταθεί με ένα β κέρδος για τελικό heading alignment.

Βήμα 4.2: Δημιουργία Python ROS Node

Δημιούργησε το αρχείο:

```
cd ~/catkin_ws/src
mkdir -p turtlebot3_controller/scripts
cd turtlebot3_controller/scripts
touch turtlebot3_motion_controller.py
chmod +x turtlebot3_motion_controller.py
```

Βήμα 4.3: Δομή Κώδικα (Skeleton)

Ο κώδικας πρέπει να περιλαμβάνει τα εξής βασικά στοιχεία:

```
#!/usr/bin/env python3
import rospy
import math
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Twist
```

```

from tf.transformations import euler_from_quaternion

class MotionController:
    def __init__(self):
        rospy.init_node("turtlebot3_motion_controller")
        # Subscriber: /odom -> callback
        # Publisher: /cmd_vel -> Twist
        # Gains, waypoints, state variables

    def odom_callback(self, msg):
        # Εξαγωγή x, y, yaw από Odometry
        # quaternion -> euler conversion

    def go_to_goal(self, x_goal, y_goal):
        # Υπολογισμός rho, alpha
        # Υπολογισμός v, omega
        # Publish Twist
        # Loop μέχρι rho < threshold

    def run(self):
        # Iterate waypoints
        # Κάλεσε go_to_goal() για κάθε ένα

if __name__ == "__main__":
    controller = MotionController()
    controller.run()

```

Βήμα 4.4: Σημαντικές Λεπτομέρειες Υλοποίησης

Quaternion σε Euler: Το /odom δίνει orientation σε quaternion. Χρειάζεσαι τη μετατροπή σε yaw (γωνία psi):

```

orientation = msg.pose.pose.orientation
quaternion = [orientation.x, orientation.y, orientation.z, orientation.w]
_, _, yaw = euler_from_quaternion(quaternion)

```

Angle wrapping: Κανονικοποίησε τη γωνιακή διαφορά alpha στο $[-\pi, \pi]$:

```

alpha = math.atan2(math.sin(alpha), math.cos(alpha))

```

Rate control: Χρησιμοποίησε `rospy.Rate(10)` για 10 Hz control loop.

Stop command: Όταν φτάνεις σε waypoint, στείλε `Twist()` (μηδενικές ταχύτητες) πριν συνεχίσεις στο επόμενο.

Βήμα 5.1: Εκτέλεση Controller

Χρειάζεσαι **2 terminals** ταυτόχρονα:

```
# Terminal 1: Ξεκίνα Gazebo
export TURTLEBOT3_MODEL=waffle
roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch

# Terminal 2: Τρέξε τον controller
cd ~/catkin_ws/src/turtlebot3_controller/scripts
python3 turtlebot3_motion_controller.py
```

Βήμα 5.2: Debugging Tips

Αν κάτι δεν δουλεύει, χρησιμοποίησε:

```
# Δες τι στέλνει ο controller:
rostopic echo /cmd_vel

# Δες τη θέση του ρομπότ:
rostopic echo /odom

# Δες τα ενεργά nodes:
roslaunch turtlebot3_gazebo turtlebot3_gazebo_rviz.launch
```

Βήμα 5.3: Τι να Ελέγξεις

- [] Το ρομπότ κινείται προς το 1ο waypoint [2.0, 2.0]
- [] Σταματάει κοντά στο waypoint ($\rho < 0.1$)
- [] Συνεχίζει στο 2ο waypoint [4.0, 4.0]
- [] Ολοκληρώνει όλα τα waypoints μέχρι [8.0, 8.0]
- [] Σταματάει τελείως μετά το τελευταίο waypoint
- [] Εκτυπώνει μηνύματα κατάστασης (πχ "Heading to waypoint 1...")
- [] Ο κώδικας έχει επεξηγηματικά σχόλια
- [] Καμία σύγκρουση ή ανεπιθύμητη συμπεριφορά

Βήμα 5.4: Προετοιμασία Αποστολής Email

Ετοίμασε τα εξής για αποστολή:

Πεδίο	Τιμή
Προς (To):	gkarras@uth.gr
Από (From):	Το ΙΔΡΥΜΑΤΙΚΟ σου email μόνο!

Θέμα:	Εργαστηριακό Θέμα - Ενότητα Α - [Ονοματεπώνυμο] - [ΑΜ]
Συνημμένο:	turtlebot3_motion_controller.py
Σώμα email:	Ονοματεπώνυμο + Αριθμός Μητρώου

ΠΡΟΣΟΧΗ: Χρησιμοποίησε **ΜΟΝΟ** το **ιδρυματικό σου email!** Emails από gmail/yahoo κλπ δεν γίνονται αποδεκτά.

ΤΕΛΙΚΟ CHECKLIST ΠΡΙΝ ΤΗΝ ΑΠΟΣΤΟΛΗ

- ☐ Ο κώδικας τρέχει χωρίς errors
- ☐ Το ρομπότ φτάνει σε ΟΛΑ τα waypoints
- ☐ Το αρχείο ονομάζεται turtlebot3_motion_controller.py
- ☐ Υπάρχουν αρκετά επεξηγηματικά σχόλια
- ☐ Το email στέλνεται ΑΠΟ ιδρυματικό email
- ☐ Το email στέλνεται ΠΡΟΣ gkarras@uth.gr
- ☐ Περιλαμβάνονται ονοματεπώνυμο + ΑΜ
- ☐ Ετοιμότητα για προφορική εξέταση στο εργαστήριο

Καλή Επιτυχία!